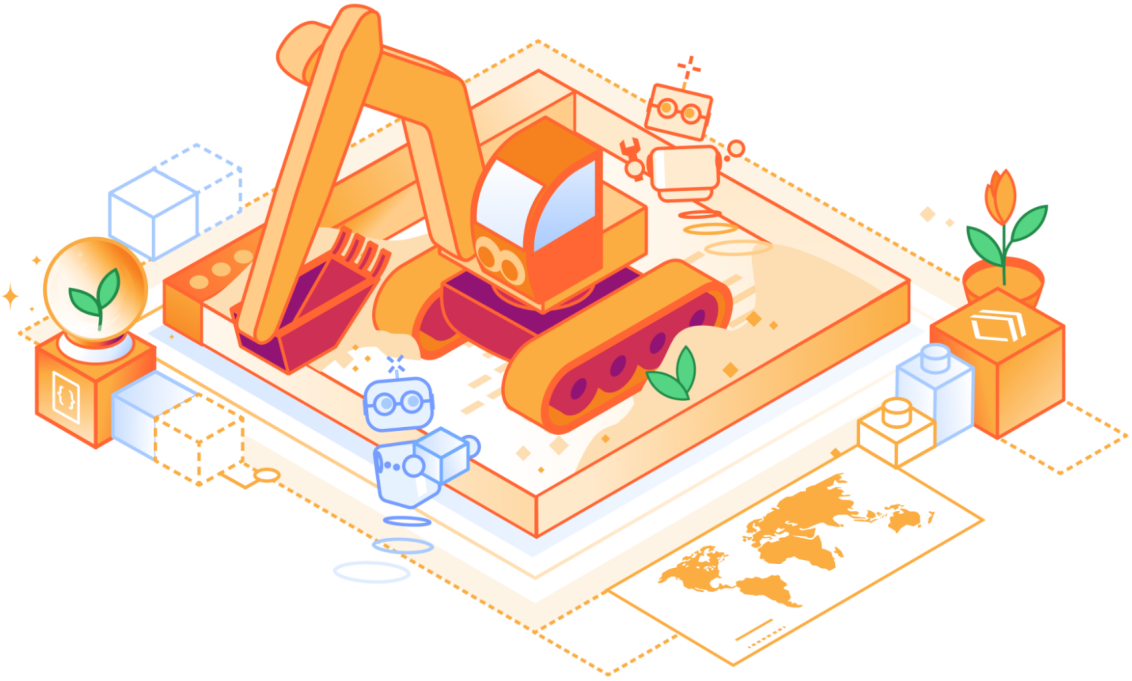
Code Mode: give agents an entire API in 1,000 tokens

2026-02-20

Matt Carey

6 min read



Model Context Protocol (MCP) has become the standard way for AI agents to use external tools. But there is a tension at its core: agents need many tools to do useful work, yet every tool added fills the model's context window, leaving less room for the actual task.

Code Mode is a technique we first introduced for reducing context window usage during agent tool use. Instead of describing every operation as a

separate tool, let the model write code against a typed SDK and execute the code safely in a Dynamic Worker Loader. The code acts as a compact plan. The model can explore tool operations, compose multiple calls, and return just the data it needs. Anthropic independently explored the same pattern in their Code Execution with MCP post.

Today we are introducing a new MCP server for the entire Cloudflare API — from DNS and Zero Trust to Workers and R2 — that uses Code Mode. With just two tools, search() and execute(), the server is able to provide access to the entire Cloudflare API over MCP, while consuming only around 1,000 tokens. The footprint stays fixed, no matter how many API endpoints exist.

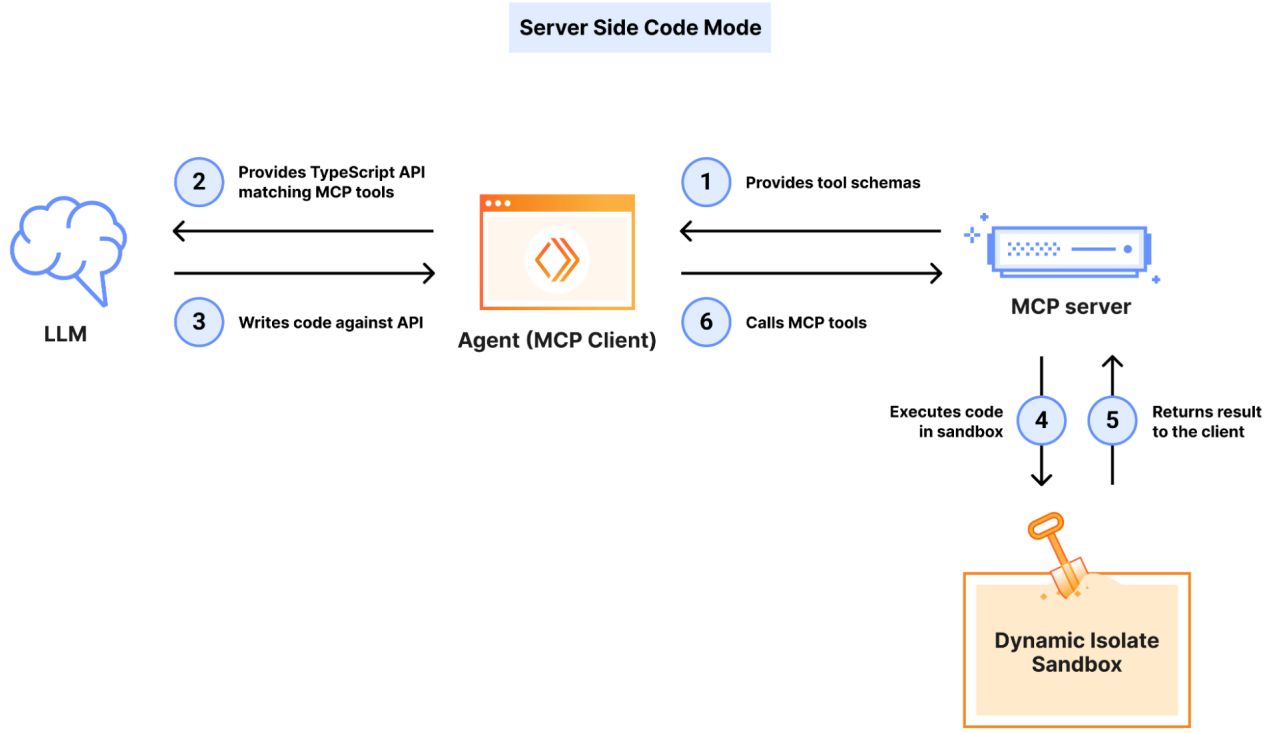
For a large API like the Cloudflare API, Code Mode reduces the number of input tokens used by 99.9%. An equivalent MCP server without Code Mode would consume 1.17 million tokens — more than the entire context window of the most advanced foundation models.

Approach	Tools	Token cost	Context used (200k)
Raw OpenAPI Spec in prompt	--	~2,000,000	977%
Native MCP (full schemas)	2,594	1,170,523	585%
Native MCP (required params only)	2,594	244,047	122%
Code Mode	2	1,069	0.5%

Code mode savings vs native MCP, measured with tiktoken

You can start using this new Cloudflare MCP server today. And we are also open-sourcing a new Code Mode SDK in the Cloudflare Agents SDK, so you can use the same approach in your own MCP servers and AI Agents.

Server-side Code Mode



This new MCP server applies Code Mode server-side. Instead of thousands of tools, the server exports just two: `search()` and `execute()`. Both are powered by Code Mode. Here is the full tool surface area that gets loaded into the model context:

```
[
  {
    "name": "search",
    "description": "Search the Cloudflare OpenAPI spec. All $refs are pre-resolved
inline.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "code": {
          "type": "string",
          "description": "JavaScript async arrow function to search the OpenAPI spec"
        }
      },
      "required": ["code"]
    }
  },
  {
    "name": "execute",
    "description": "Execute JavaScript code against the Cloudflare API.",
    "inputSchema": {
```

```
        "type": "object",
        "properties": {
          "code": {
            "type": "string",
            "description": "JavaScript async arrow function to execute"
          }
        },
        "required": ["code"]
      }
    }
  ]
```

To discover what it can do, the agent calls `search()`. It writes JavaScript against a typed representation of the OpenAPI spec. The agent can filter endpoints by product, path, tags, or any other metadata and narrow thousands of endpoints to the handful it needs. The full OpenAPI spec never enters the model context. The agent only interacts with it through code.

When the agent is ready to act, it calls `execute()`. The agent writes code that can make Cloudflare API requests, handle pagination, check responses, and chain operations together in a single execution.

Both tools run the generated code inside a Dynamic Worker isolate — a lightweight V8 sandbox with no file system, no environment variables to leak through prompt injection and external fetches disabled by default. Outbound requests can be explicitly controlled with outbound fetch handlers when needed.

Example: Protecting an origin from DDoS attacks

Suppose a user tells their agent: "protect my origin from DDoS attacks." The agent's first step is to consult documentation. It might call the Cloudflare Docs MCP Server, use a Cloudflare Skill, or search the web directly. From the docs it learns: put Cloudflare WAF and DDoS protection rules in front of the origin.

Step 1: Search for the right endpoints The `search` tool gives the model a

`spec` object: the full Cloudflare OpenAPI spec with all `$refs` pre-resolved. The model writes JavaScript against it. Here the agent looks for WAF and ruleset endpoints on a zone:

```
async () => {
  const results = [];
  for (const [path, methods] of Object.entries(spec.paths)) {
    if (path.includes('/zones/') &&
        (path.includes('firewall/waf') || path.includes('rulesets'))) {
      for (const [method, op] of Object.entries(methods)) {
        results.push({ method: method.toUpperCase(), path, summary: op.summary });
      }
    }
  }
  return results;
}
```

The server runs this code in a Workers isolate and returns:

```
[
  { "method": "GET",    "path": "/zones/{zone_id}/firewall/waf/packages",
"summary": "List WAF packages" },
  { "method": "PATCH",  "path": "/zones/{zone_id}/firewall/waf/packages/{package_id}",
"summary": "Update a WAF package" },
  { "method": "GET",    "path": "/zones/{zone_id}/firewall/waf/packages/{package_id}/
rules", "summary": "List WAF rules" },
  { "method": "PATCH",  "path": "/zones/{zone_id}/firewall/waf/packages/{package_id}/rules/
{rule_id}", "summary": "Update a WAF rule" },
  { "method": "GET",    "path": "/zones/{zone_id}/rulesets",
"summary": "List zone rulesets" },
  { "method": "POST",   "path": "/zones/{zone_id}/rulesets",
"summary": "Create a zone ruleset" },
  { "method": "GET",    "path": "/zones/{zone_id}/rulesets/phases/{ruleset_phase}/
entrypoint", "summary": "Get a zone entry point ruleset" },
  { "method": "PUT",    "path": "/zones/{zone_id}/rulesets/phases/{ruleset_phase}/
entrypoint", "summary": "Update a zone entry point ruleset" },
  { "method": "POST",   "path": "/zones/{zone_id}/rulesets/{ruleset_id}/rules",
"summary": "Create a zone ruleset rule" },
  { "method": "PATCH",  "path": "/zones/{zone_id}/rulesets/{ruleset_id}/rules/{rule_id}",
"summary": "Update a zone ruleset rule" }
]
```

The full Cloudflare API spec has over 2,500 endpoints. The model narrowed that to the WAF and ruleset endpoints it needs, without any of the spec

entering the context window.

The model can also drill into a specific endpoint's schema before calling it. Here it inspects what phases are available on zone rulesets:

```
async () => {
  const op = spec.paths['/zones/{zone_id}/rulesets']?.get;
  const items = op?.responses?.['200']?.content?.['application/json']?.schema;
  // Walk the schema to find the phase enum
  const props = items?.allOf?.[1]?.properties?.result?.items?.allOf?.[1]?.properties;
  return { phases: props?.phase?.enum };
}
```

```
{
  "phases": [
    "ddos_l4", "ddos_l7",
    "http_request_firewall_custom", "http_request_firewall_managed",
    "http_response_firewall_managed", "http_ratelimit",
    "http_request_redirect", "http_request_transform",
    "magic_transit", "magic_transit_managed"
  ]
}
```

The agent now knows the exact phases it needs: `ddos_l7` for DDoS protection and `http_request_firewall_managed` for WAF.

Step 2: Act on the API The agent switches to using `execute`. The sandbox gets a `cloudflare.request()` client that can make authenticated calls to the Cloudflare API. First the agent checks what rulesets already exist on the zone:

```
async () => {
  const response = await cloudflare.request({
    method: "GET",
    path: `/zones/${zoneId}/rulesets`
  });
  return response.result.map(rs => ({
    name: rs.name, phase: rs.phase, kind: rs.kind
  }));
}
```

```
[
  { "name": "DDoS L7",          "phase": "ddos_l7",                        "kind":
```

```
  "managed" },
    { "name": "Cloudflare Managed","phase": "http_request_firewall_managed", "kind":
  "managed" },
    { "name": "Custom rules",     "phase": "http_request_firewall_custom",   "kind": "zone" }
  ]
```

The agent sees that managed DDoS and WAF rulesets already exist. It can now chain calls to inspect their rules and update sensitivity levels in a single execution:

```
  async () => {
    // Get the current DDoS L7 entrypoint ruleset
    const ddos = await cloudflare.request({
      method: "GET",
      path: `/zones/${zoneId}/rulesets/phases/ddos_l7/entrypoint`
    });

    // Get the WAF managed ruleset
    const waf = await cloudflare.request({
      method: "GET",
      path: `/zones/${zoneId}/rulesets/phases/http_request_firewall_managed/entrypoint`
    });
  }
```

This entire operation, from searching the spec and inspecting a schema to listing rulesets and fetching DDoS and WAF configurations, took four tool calls.

The Cloudflare MCP server

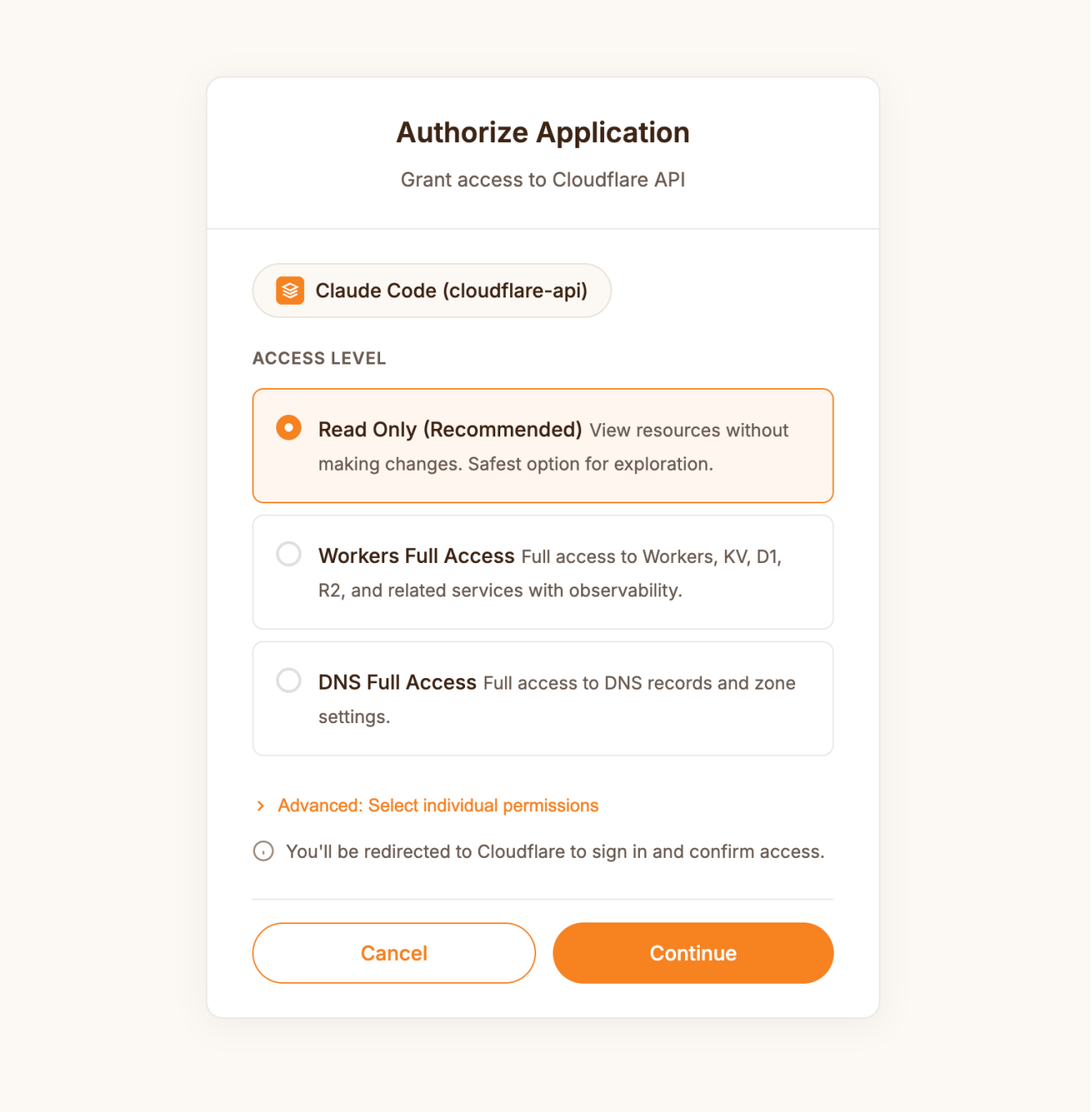
We started with MCP servers for individual products. Want an agent that manages DNS? Add the DNS MCP server. Want Workers logs? Add the Workers Observability MCP server. Each server exported a fixed set of tools that mapped to API operations. This worked when the tool set was small, but the Cloudflare API has over 2,500 endpoints. No collection of hand-maintained servers could keep up.

The Cloudflare MCP server simplifies this. Two tools, roughly 1,000 tokens, and coverage of every endpoint in the API. When we add new products, the

same `search()` and `execute()` code paths discover and call them — no new tool definitions, no new MCP servers. It even has support for the GraphQL Analytics API.

Our MCP server is built on the latest MCP specifications. It is OAuth 2.1 compliant, using Workers OAuth Provider to downscope the token to selected permissions approved by the user when connecting. The agent only gets the capabilities the user explicitly granted.

For developers, this means you can use a simple agent loop and still give your agent access to the full Cloudflare API with built-in progressive capability discovery.



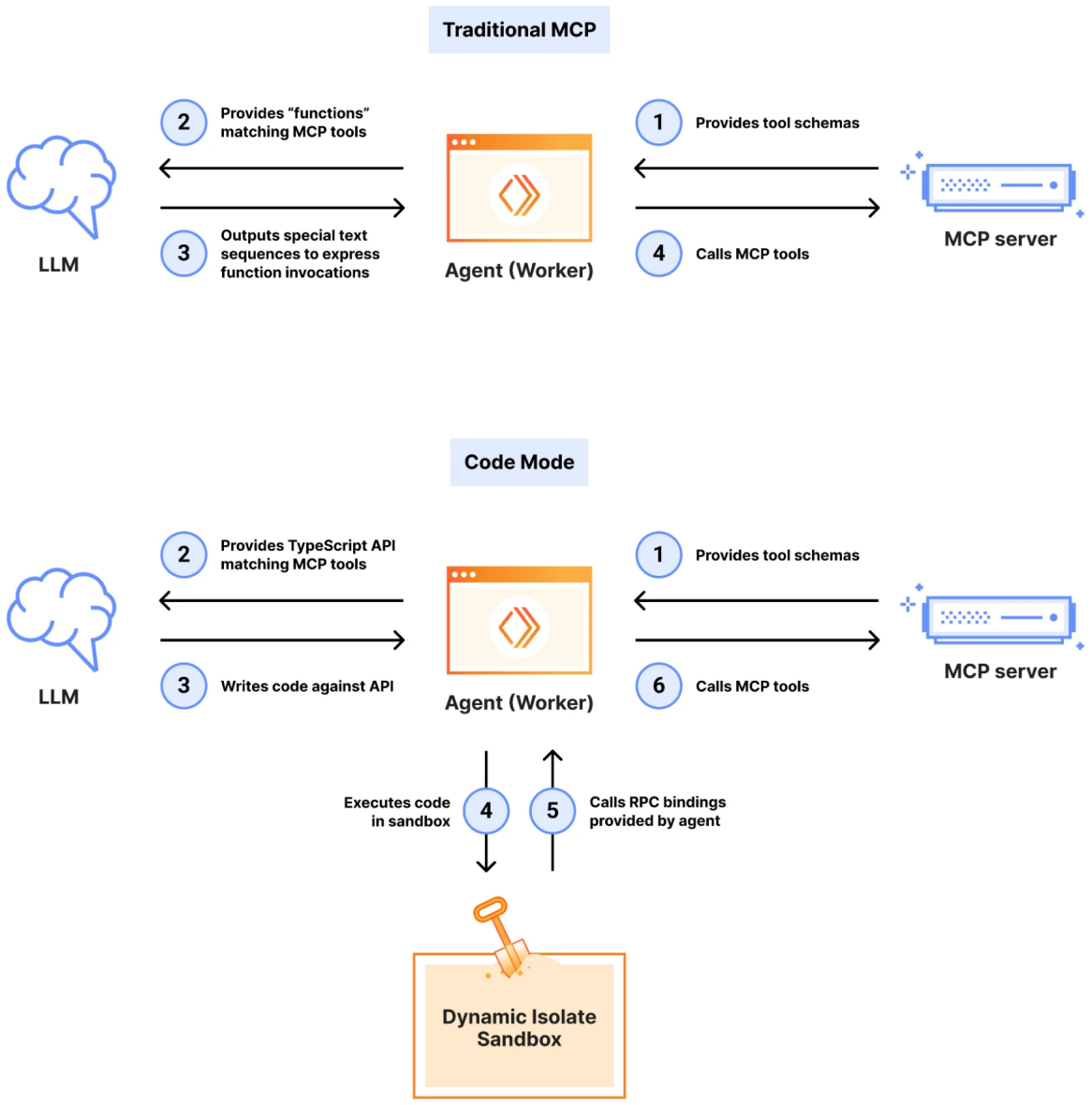
Comparing approaches to context reduction

Several approaches have emerged to reduce how many tokens MCP tools consume:

Client-side Code Mode was our first experiment. The model writes TypeScript against typed SDKs and runs it in a Dynamic Worker Loader on the client. The tradeoff is that it requires the agent to ship with secure sandbox access. Code Mode is implemented in Goose and Anthropics Claude SDK as Programmatic Tool Calling.

Command-line interfaces are another path. CLIs are self-documenting and reveal capabilities as the agent explores. Tools like OpenClaw and Moltworker convert MCP servers into CLIs using MCPorter to give agents progressive disclosure. The limitation is obvious: the agent needs a shell, which not every environment provides and which introduces a much broader attack surface than a sandboxed isolate.

Dynamic tool search, as used by Anthropic in Claude Code, surfaces a smaller set of tools hopefully relevant to the current task. It shrinks context use but now requires a search function that must be maintained and evaluated, and each matched tool still uses tokens.



Each approach solves a real problem. But for MCP servers specifically, server-side Code Mode combines their strengths: fixed token cost regardless of API size, no modifications needed on the agent side, progressive discovery built in, and safe execution inside a sandboxed isolate. The agent just calls two tools with code. Everything else happens on the server.

Get started today

The Cloudflare MCP server is available now. Point your MCP client at the server URL and you'll be redirected to Cloudflare to authorize and select the permissions to grant to your agent. Add this config to your MCP client:

```
{
  "mcpServers": {
    "cloudflare-api": {
      "url": "https://mcp.cloudflare.com/mcp"
    }
  }
}
```

For CI/CD, automation, or if you prefer managing tokens yourself, create a Cloudflare API token with the permissions you need. Both user tokens and account tokens are supported and can be passed as bearer tokens in the `Authorization` header.

More information on different MCP setup configurations can be found at the Cloudflare MCP repository.

Looking forward

Code Mode solves context costs for a single API. But agents rarely talk to one service. A developer's agent might need the Cloudflare API alongside GitHub, a database, and an internal docs server. Each additional MCP server brings the same context window pressure we started with.

Cloudflare MCP Server Portals let you compose multiple MCP servers

behind a single gateway with unified auth and access control. We are building a first-class Code Mode integration for all your MCP servers, and exposing them to agents with built-in progressive discovery and the same fixed-token footprint, regardless of how many services sit behind the gateway.

Cloudflare's connectivity cloud protects entire corporate networks, helps customers build Internet-scale applications efficiently, accelerates any website or Internet application, wards off DDoS attacks, keeps hackers at bay, and can help you on your journey to Zero Trust.

Visit 1.1.1.1 from any device to get started with our free app that makes your Internet faster and safer.

To learn more about our mission to help build a better Internet, start here. If you're looking for a new career direction, check out our open positions.

Discuss on Hacker News

Developers Developer Platform AI Workers AI Cloudflare Workers Optimization Open Source

---

Follow on X

Matt Carey | mattzcarey
Cloudflare | @cloudflare

RELATED POSTS

February 13, 2026 3:00 PM

Shedding old code with ecdysis: graceful restarts for Rust services at Cloudflare

ecdysis is a Rust library enabling zero-downtime upgrades for network services. After five years protecting millions of connections at Cloudflare, it's now open source....

By Manuel Olguín Muñoz

Rust, Open Source, Infrastructure, Engineering, Edge, Developers, Developer Platform, Application Services, Rust

February 12, 2026 3:03 PM

Introducing Markdown for Agents

The way content is discovered online is shifting, from traditional search engines to AI agents that need structured data from a Web built for humans. It's time to consider not just human visitors, but start to treat agents as first-class citizens. Markdown for Agents automatically converts any HTML page requested from our network to markdown....

By Celso Martinho, Will Allen

AI, Agents, Developers, Developer Platform

February 05, 2026 3:00 PM

2025 Q4 DDoS threat report: A record-setting 31.4 Tbps attack caps a year of massive DDoS assaults

The number of DDoS attacks more than doubled in 2025. The network layer is under particular threat as hyper-volumetric attacks grew 700%....

By Omer Yoachimik, Jorge Pacheco, Cloudforce One

DDoS Reports, DDoS, Cloudforce One, Security, Advanced DDoS, AI

January 30, 2026 6:01 PM

Google’s AI advantage: why crawler separation is the only path to a fair Internet

Google's dual-purpose crawler creates an unfair AI advantage. To protect publishers and foster competition, the UK’s Competition and Markets Authority must mandate crawler separation for search and AI....

By Maria Palmieri, Sebastian Hufnagel

AI, AI Bots, Google, Legal, Policy & Legal

© 2026 Cloudflare, Inc. | Privacy Policy | Terms of Use | Report Security Issues | Cookie Preferences| Trademark